Name: Nkoka Mashaba

Student Number: ST10458688

Module Name: Advanced Databases

Module Code: ADDB6311

Due Date: 8 May 2026

Practical Assignment 2

Question 1

CREATE TABLE CUSTOMER (CUSTOMER_ID NUMBER PRIMARY KEY, FIRST_NAME VARCHAR2(50), SURNAME VARCHAR2(50), ADDRESS VARCHAR2(100), CONTACT_NUMBER VARCHAR2(15), EMAIL VARCHAR2(100));

	CUSTOMER_ID	FIRST_NAME	SURNAME	ADDRESS	CONTACT_NUMBER	EMAIL
1	11011	Jack	Smith	18 Water Rd	0877277521	jsmith@isat.com
2	11012	Pat	Hendricks	22 Water Rd	0863257857	ph@mcom.co.za
3	11013	Andre	Clark	101 Summer Lane	0834567891	aclark@mcom.co.za
4	11014	Kevin	Jones	55 Mountain way	0612547895	kj@isat.co.za
5	11015	Lucy	Williams	5 Main rd	0827238521	lw@mcal.co.za

CREATE TABLE EMPLOYEE (EMPLOYEE_ID VARCHAR2(10) PRIMARY KEY, FIRST_NAME VARCHAR2(50), SURNAME VARCHAR2(50), CONTACT_NUMBER VARCHAR2(15), ADDRESS VARCHAR2(100), EMAIL VARCHAR2(100));

	EMPLOYEE_ID	FIRST_NAME	SURNAME	CONTACT_NUMBER	ADDRESS	EMAIL
1	emp101	Jeff	Davis	0877277521	10 main road	jand@isat.com
2	emp102	Kevin	Marks	0837377522	18 water road	km@isat.com
3	emp103	Adanya	Andrews	0817117523	21 circle lane	aa@isat.com
4	emp104	Adebayo	Dryer	0797215244	1 sea road	aryer@isat.com
5	emp105	Xolani	Samson	0827122255	12 main road	xosam@isat.com

CREATE TABLE DONATOR (DONATOR_ID NUMBER PRIMARY KEY, FIRST_NAME VARCHAR2(50), SURNAME VARCHAR2(50), CONTACT_NUMBER VARCHAR2(15), EMAIL VARCHAR2(100));

	DONATOR_ID	FIRST_NAME	SURNAME	CONTACT_NUMBER	EMAIL
1	20111	Jeff	Watson	0827172250	jwatson@ymail.com
2	20112	Stephen	Jones	0837865670	joness@ymail.com
3	20113	James	Joe	0878978650	jj@isat.com
4	20114	Kelly	Ross	0826575650	kross@gsat.com
5	20115	Abraham	Clark	0797656430	aclark@ymail.com

CREATE TABLE DONATION (DONATION_ID NUMBER PRIMARY KEY, DONATOR_ID NUMBER, DONATION VARCHAR2(100), PRICE NUMBER(10, 2), DONATION_DATE DATE, CONSTRAINT FK_DONATOR FOREIGN KEY (DONATOR_ID) REFERENCES DONATOR(DONATOR_ID));

	DONATION_ID	DONATOR_ID	DONATION	PRICE	DONATION_DATE
1	7111	20111	KIC Fridge	599	01-MAY-24
2	7112	20112	Samsung 42inch LCD	1299	03-MAY-24
3	7113	20113	Sharp Microwave	1599	03-MAY-24
4	7114	20115	6 Seat Dining room table	799	05-MAY-24
5	7115	20114	Lazyboy Sofa	1199	07-MAY-24
6	7116	20113	JVC Surround Sound System	179	09-MAY-24

CREATE TABLE DELIVERY (DELIVERY_ID NUMBER PRIMARY KEY, DELIVERY_NOTES VARCHAR2(200), DISPATCH_DATE DATE, DELIVERY_DATE DATE);

	DELIVERY_ID	DELIVERY_NOTES	DISPATCH_DATE	DELIVERY_DATE
1	511	Double packaging requested	10-MAY-24	15-MAY-24
2	512	Delivery to work address	12-MAY-24	15-MAY-24
3	513	Signature required	12-MAY-24	17-MAY-24
4	514	No notes	12-MAY-24	15-MAY-24
5	515	Birthday present wrapping required	18-MAY-24	19-MAY-24
6	516	Delivery to work address	20-MAY-24	25-MAY-24

CREATE TABLE INVOICE (INVOICE_NUM NUMBER PRIMARY KEY, CUSTOMER_ID NUMBER, INVOICE_DATE DATE, EMPLOYEE_ID VARCHAR2(10), DONATION_ID NUMBER, DELIVERY_ID NUMBER, CONSTRAINT FK_CUST FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID), CONSTRAINT FK_EMP FOREIGN KEY (EMPLOYEE_ID) REFERENCES EMPLOYEE(EMPLOYEE_ID), CONSTRAINT FK_DON FOREIGN KEY (DONATION_ID) REFERENCES DONATION(DONATION_ID), CONSTRAINT FK_DEL FOREIGN KEY (DELIVERY_ID) REFERENCES DELIVERY(DELIVERY_ID));

	INVOICE_NUM	CUSTOMER_ID	INVOICE_DATE	EMPLOYEE_ID	DONATION_ID	DELIVERY_ID
1	8111	11011	15-MAY-24	emp103	7111	511
2	8112	11013	15-MAY-24	emp101	7114	512
3	8113	11012	17-MAY-24	emp101	7112	513
4	8114	11015	17-MAY-24	emp102	7113	514
5	8115	11011	17-MAY-24	emp102	7115	515
6	8116	11015	18-MAY-24	emp103	7116	516

CREATE TABLE RETURNS (RETURN_ID VARCHAR2(10) PRIMARY KEY, RETURN_DATE DATE, REASON VARCHAR2(200), CUSTOMER_ID NUMBER, DONATION_ID NUMBER, EMPLOYEE_ID VARCHAR2(10), CONSTRAINT FK_RET_CUST FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID), CONSTRAINT FK_RET_DON FOREIGN KEY (DONATION_ID) REFERENCES DONATION(DONATION_ID), CONSTRAINT FK_RET_EMP FOREIGN KEY (EMPLOYEE_ID) REFERENCES EMPLOYEE(EMPLOYEE_ID));

	RETURN_ID	RETURN_DATE	REASON	CUSTOMER_ID	DONATION_ID	EMPLOYEE_ID
1	ret001	25-MAY-24	Customer not satisfied with product	11011	7116	emp101
2	ret002	25-MAY-24	Product had broken section	11013	7114	emp103

Question 2

```
SELECT
   c.FIRST_NAME || ', ' || c.SURNAME AS CUSTOMER,
   i.EMPLOYEE_ID,
   d.DELIVERY_NOTES,
   dn.DONATION,
   i.INVOICE_NUM,
   i.INVOICE_DATE
FROM INVOICE i
JOIN CUSTOMER c ON i.CUSTOMER_ID = c.CUSTOMER_ID
JOIN DELIVERY d ON i.DELIVERY_ID = d.DELIVERY_ID
JOIN DONATION dn ON i.DONATION_ID = dn.DONATION_ID
WHERE i.INVOICE_DATE < TO_DATE('18-MAY-2024', 'DD-MON-YYYY');
```

	CUSTOMER	EMPLOYEE_ID	DELIVERY_NOTES	DONATION	INVOICE_NUM	INVOICE_DATE
1	Jack, Smith	emp103	Double packaging requested	KIC Fridge	8111	15-MAY-24
2	Pat, Hendricks	emp101	Signature required	Samsung 42inch LCD	8113	17-MAY-24
3	Lucy, Williams	emp102	No notes	Sharp Microwave	8114	17-MAY-24
4	Andre, Clark	emp101	Delivery to work address	6 Seat Dining room table	8112	15-MAY-24
5	Jack, Smith	emp102	Birthday present wrapping required	Lazyboy Sofa	8115	17-MAY-24

Question 3

CREATE TABLE Funding (funding_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, funder VARCHAR2(100), funding_amount NUMBER(12, 2));

	FUNDING_ID	FUNDER	FUNDING_AMOUNT
1	1	Global Aid	50000

Question 4

```
SET SERVEROUTPUT ON;

DECLARE
  CURSOR ret_cur IS
    SELECT c.FIRST_NAME, c.SURNAME, d.DONATION, d.PRICE, r.REASON
    FROM RETURNS r
    JOIN CUSTOMER c ON r.CUSTOMER_ID = c.CUSTOMER_ID
    JOIN DONATION d ON r.DONATION_ID = d.DONATION_ID;
BEGIN
  FOR rec IN ret_cur LOOP
    DBMS_OUTPUT.PUT_LINE('CUSTOMER: ' || rec.FIRST_NAME || ', ' || rec.SURNAME);
    DBMS_OUTPUT.PUT_LINE('DONATION PURCHASED: ' || rec.DONATION);
    DBMS_OUTPUT.PUT_LINE('PRICE: ' || rec.PRICE);
    DBMS_OUTPUT.PUT_LINE('RETURN REASON: ' || rec.REASON);
    DBMS_OUTPUT.PUT_LINE('-----------------------------------');
  END LOOP;
END;
```

```
PL/SQL procedure successfully completed.
```

Question 5

```
DECLARE
   v_cust_name VARCHAR2(100);
   v_emp_name  VARCHAR2(100);
   v_donation  VARCHAR2(100);
   v_dispatch  DATE;
   v_delivery  DATE;
   v_days      NUMBER;
BEGIN
   SELECT c.FIRST_NAME || ' ' || c.SURNAME, e.FIRST_NAME || ' ' || e.SURNAME,
          dn.DONATION, dl.DISPATCH_DATE, dl.DELIVERY_DATE,
          (dl.DELIVERY_DATE - dl.DISPATCH_DATE)
   INTO v_cust_name, v_emp_name, v_donation, v_dispatch, v_delivery, v_days
   FROM INVOICE i
   JOIN CUSTOMER c ON i.CUSTOMER_ID = c.CUSTOMER_ID
   JOIN EMPLOYEE e ON i.EMPLOYEE_ID = e.EMPLOYEE_ID
   JOIN DONATION dn ON i.DONATION_ID = dn.DONATION_ID
   JOIN DELIVERY dl ON i.DELIVERY_ID = dl.DELIVERY_ID
   WHERE c.CUSTOMER_ID = 11013 AND ROWNUM = 1;

   DBMS_OUTPUT.PUT_LINE('CUSTOMER: ' || v_cust_name);
   DBMS_OUTPUT.PUT_LINE('EMPLOYEE: ' || v_emp_name);
```

```
  DBMS_OUTPUT.PUT_LINE('DAYS TO DELIVERY: ' || v_days);
END;
```

```
CUSTOMER: Andre Clark
EMPLOYEE: Jeff Davis
DAYS TO DELIVERY: 3


PL/SQL procedure successfully completed.
```

Question 6

```
BEGIN
  FOR rec IN (SELECT FIRST_NAME, SURNAME, SUM(PRICE) as total
        FROM INVOICE i
        JOIN CUSTOMER c ON i.CUSTOMER_ID = c.CUSTOMER_ID
        JOIN DONATION d ON i.DONATION_ID = d.DONATION_ID
        GROUP BY FIRST_NAME, SURNAME)
  LOOP
    DBMS_OUTPUT.PUT_LINE('FIRST NAME: ' || rec.FIRST_NAME);
    DBMS_OUTPUT.PUT_LINE('SURNAME: ' || rec.SURNAME);
    IF rec.total >= 1500 THEN
      DBMS_OUTPUT.PUT_LINE('AMOUNT: R ' || rec.total || ' (*)');
    ELSE
      DBMS_OUTPUT.PUT_LINE('AMOUNT: R ' || rec.total);
    END IF;
    DBMS_OUTPUT.PUT_LINE('-----------------------------------');
  END LOOP;
```

END;

```
FIRST NAME: Jack
SURNAME: Smith
AMOUNT: R 1798 (*)
----------------------------------
FIRST NAME: Pat
SURNAME: Hendricks
AMOUNT: R 1299
----------------------------------
FIRST NAME: Lucy
SURNAME: Williams
AMOUNT: R 1778 (*)
----------------------------------
FIRST NAME: Andre
SURNAME: Clark
AMOUNT: R 799
----------------------------------


PL/SQL procedure successfully completed.
```

Question 7

7.1 DECLARE

-- Using %TYPE to ensure variable matches database column definition

v_emp_email EMPLOYEE.EMAIL%TYPE;

BEGIN

SELECT EMAIL INTO v_emp_email FROM EMPLOYEE WHERE EMPLOYEE_ID = 'emp101';

DBMS_OUTPUT.PUT_LINE('Employee Email: ' || v_emp_email);

END;

```
Employee Email: jand@isat.com


PL/SQL procedure successfully completed.
```

7.2

```
DECLARE
-- Using %ROWTYPE to capture a full record structure
  v_donator_rec DONATOR%ROWTYPE;
BEGIN
  SELECT * INTO v_donator_rec FROM DONATOR WHERE DONATOR_ID = 20111;
  DBMS_OUTPUT.PUT_LINE('Donator: ' || v_donator_rec.FIRST_NAME || ' ' ||
v_donator_rec.SURNAME);
END;
```

```
Donator: Jeff Watson


PL/SQL procedure successfully completed.
```

7.3

```
BEGIN
  -- This will trigger NO_DATA_FOUND if ID 999 doesn't exist
  UPDATE DONATION SET PRICE = PRICE + 10 WHERE DONATION_ID = 999;
  IF SQL%NOTFOUND THEN
    RAISE NO_DATA_FOUND;
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: The specified Donation ID was not found.');
```

END;

```
Error: The specified Donation ID was not found.


PL/SQL procedure successfully completed.
```

7.4 DECLARE

```
  e_invalid_price EXCEPTION;
  v_price NUMBER := -50; -- Test value
BEGIN
  IF v_price < 0 THEN
    RAISE e_invalid_price;
  END IF;
EXCEPTION
  WHEN e_invalid_price THEN
    DBMS_OUTPUT.PUT_LINE('Error: Donation price cannot be a negative value.');
END;
```

```
Error: Donation price cannot be a negative value.


PL/SQL procedure successfully completed.
```